

Математическое моделирование химико-технологических процессов

Лекция 8. Классификация и кластеризация данных

Виталий Алексеевич Горбунов

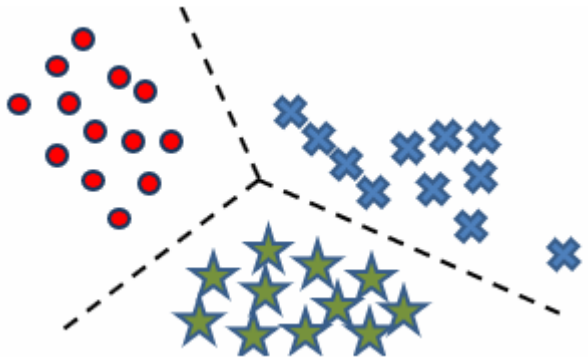
Классическое машинное обучение



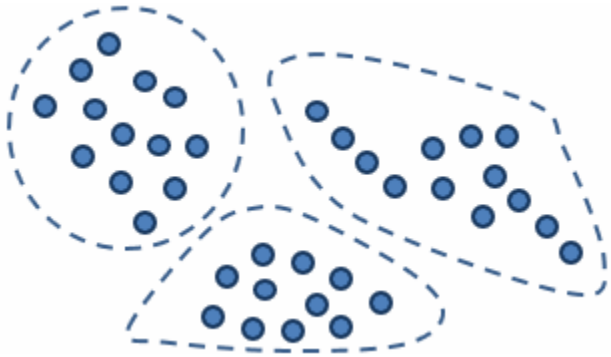
https://vas3k.ru/

С учителем vs. Без учителя

- ✓ С учителем (supervised learning): у модели есть некий «учитель», который говорит ей как правильно. «Учитель» заранее разделил (разметил) все данные, а модель учится на конкретных примерах.
  - Классификация – разделение объектов по заранее известному признаку.

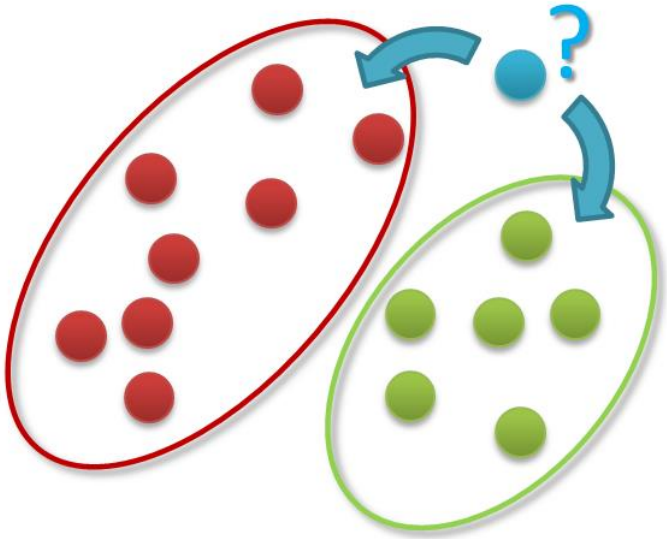


- ✓ Без учителя (unsupervised learning): данные не размечены, модель сама пытается найти любые закономерности.
  - Кластеризация – разделяет объекты по неизвестному признаку. Модель сама решает как лучше.



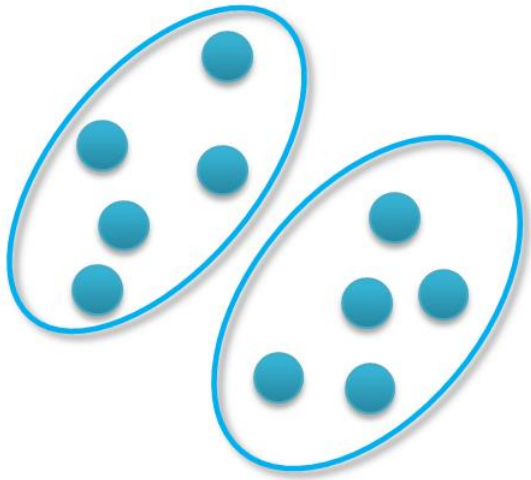
Классические методы анализа данных

Классификация



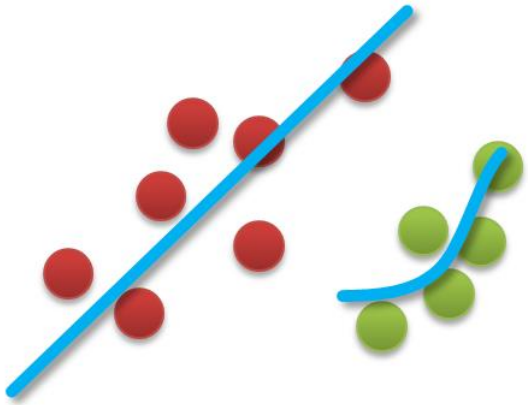
«Распредели реакции в НП по тепловому эффекту»

Кластеризация



«Распредели реакторы на НПЗ по типу (какому именно – я не знаю)»

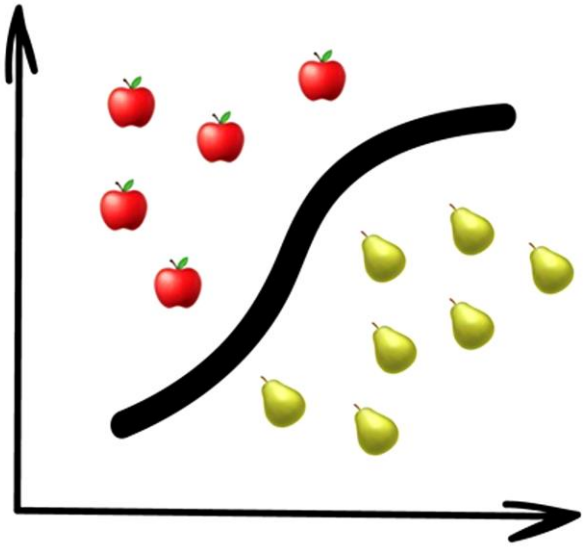
Регрессия



«Как степень конверсии вакуумного газойля зависит от высоты лифт-реактора?»

Классификация

- Наивный байесовский классификатор
- Деревья решений
- Логистическая регрессия
- kNN – k ближайших соседей



✓ Для классификации всегда нужен *учитель — размеченные данные с признаками и категориями*, которые машина будет учиться определять по этим признакам.

https://vas3k.ru/

https://tproger.ru/translations/scikit-learn-in-python

Наивный байесовский классификатор

- **Наивный байесовский классификатор** – это алгоритм классификации, *основанный на теореме Байеса* с допущением о независимости признаков, то есть, предполагается, что наличие какого-либо признака в классе не связано с наличием других признаков.

- Теорема Байеса $$P(c|x) = \frac{P(x|c)P(c)}{P(x)}$$

$P(c)$ — априорная вероятность класса C
$P(x)$ — априорная вероятность того, что признак имеет значение X
$P(c|x)$ — апостериорная вероятность класса C при значении признака X
$P(x|c)$ — правдоподобие, т.е. вероятность значения признака X в классе C

Наивный байесовский классификатор, как водичка?

- Насыщение кислородом > 18% … 1003
- Соли аммония > 12 мг/л … 364
- pH > 9 … 211
- Оксиды азота > 8 мг/л … 1716
- Ортофосфаты > 8 мг/л … 13
- Электропроводность > 6 µСм/см … 136

- Насыщение кислородом > 18% … 1829
- Соли аммония > 12 мг/л … 1710
- pH > 9 … 1191
- Оксиды азота > 8 мг/л … 985
- Ортофосфаты > 8 мг/л … 747
- Электропроводность > 6 µСм/см … 739

Хорошая вода

211 раз

Плохая вода

1191 раз

Образец воды:

pH > 9

$$P(c|x) = \frac{P(x|c)P(c)}{P(x)}$$

Плохая вода!

Наивный байесовский алгоритм

- ✓ Есть один признак или набор признаков и целевая переменная.
  На основе признаков нужно предсказать целевую переменную:

1. Преобразовать набор данных в частотную таблицу
2. Создать таблицу правдоподобия, рассчитав вероятности
3. По формуле Байеса рассчитать апостериорную вероятность каждого класса
4. Класс с наибольшей апостериорной вероятностью будет результатом прогноза.

- ✓ Отравится ли катализатор риформинга, если на переработку поступит сырье с содержанием меди и свинца более 0.3 мг/кг?

1. $\mathrm{P}(\text{да} \mid > 0.3) = \mathrm{P}(> 0.3 \mid \text{да})\mathrm{P}(\text{да})/\mathrm{P}(> 0.3)$
2. $\mathrm{P}(> 0.3 \mid \text{да}) = 3/9 = 0.33$, $\mathrm{P}(\text{да}) = \frac{9}{14} = 0.64$, $\mathrm{P}(> 0.3) = \frac{5}{14} = 0.36$
3. $\mathrm{P}(\text{да} \mid > 0.3) = 0.33 \cdot 0.64/0.36 = 0.6$
4. Прогноз: при содержании меди и свинца более 0.3 мг/кг сырья произойдет отравление катализатора риформинга.

Наивный байесовский алгоритм: + и – и применение

+ Быстрота алгоритма, в том числе многоклассового
+ Когда допущение о независимости выполняется, превосходит другие алгоритмы и требует меньший объем обучающих данных.
+ Лучше работает с категорийными признаками, чем с непрерывными.

✓ Применение:

- Классификация в режиме реального времени, потому что быстрый.
- Многоклассовый
- Рекомендательные системы

- Если значение категорийного признака, не встречалось в обучающем наборе данных, тогда модель присвоит нулевую вероятность этому значению и не сможет сделать прогноз («нулевая частота» - zero frequency). Решается с помощью сглаживания по Лапласу.
- Является хорошим классификатором, но значения спрогнозированных вероятностей не являются достаточно точными.
- Допущение о независимости признаков.
- Для непрерывных признаков предполагается нормальное распределение, что является достаточно сильным допущением.

Наивный байесовский алгоритм: scikit-learn

✓ Библиотека scikit-learn содержит три типа моделей на основе НБА:

- Gaussian. Модель используется для непрерывных признаков и предполагает, что значения признаков имеют нормальное распределение Гаусса.
- Multinomial. Модель используется для дискретных признаков. Например, сколько раз данный признак встречается в процессе.
- Bernoulli. Используется в случае двоичных дискретных признаков (могут принимать только два значения: 0 и 1 – присутствие и отсутствие признака.

```
#Импортируем библиотеку НБА с гауссовой моделью
from sklearn.naive_bayes import GaussianNB
#Импортируем библиотеку NumPy
import numpy as np

#Задаем значения признаков (x)
x= np.array([[-3,7],[1,5], [1,2], [-2,0], [2,3],
[-4,0], [-1,1], [1,1], [-2,2], [2,7], [-4,1], [-2,7]])
#Задаем соответствующие значения целевой переменной
Y = np.array([3, 3, 3, 3, 4, 3, 3, 4, 3, 4, 4, 4])

#Создаем гауссов классификатор
model = GaussianNB()

# Обучаем модель
model.fit(x, Y)

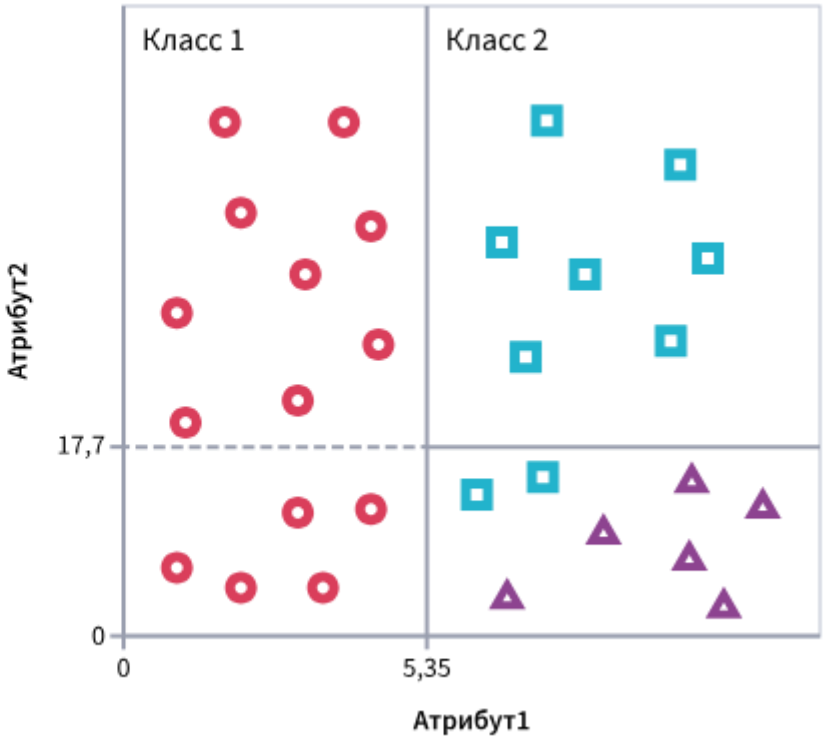
#Делаем предсказание
predicted= model.predict([[1,2],[3,4]])
print predicted

Output: ([3,4])
```

Дерево решений

- ***Дерево решений*** — классификатор, построенный на основе решающих правил вида «если, то», упорядоченных в древовидную иерархическую структуру. Это последовательное разбиение исходного множества объектов на подмножества, ассоциированные с заданными классами на основе решающих правил (условий).





Дерево решений: задача и основные понятия

- **Дано:** дано некоторое обучающее множество ***T***, содержащее объекты (примеры), каждый из которых характеризуется ***m*** признаками (атрибутами), указывающими на принадлежность объекта к определенному классу C_k.

- **Нужно:** Определить класс для другого множества по этим же атрибутам.

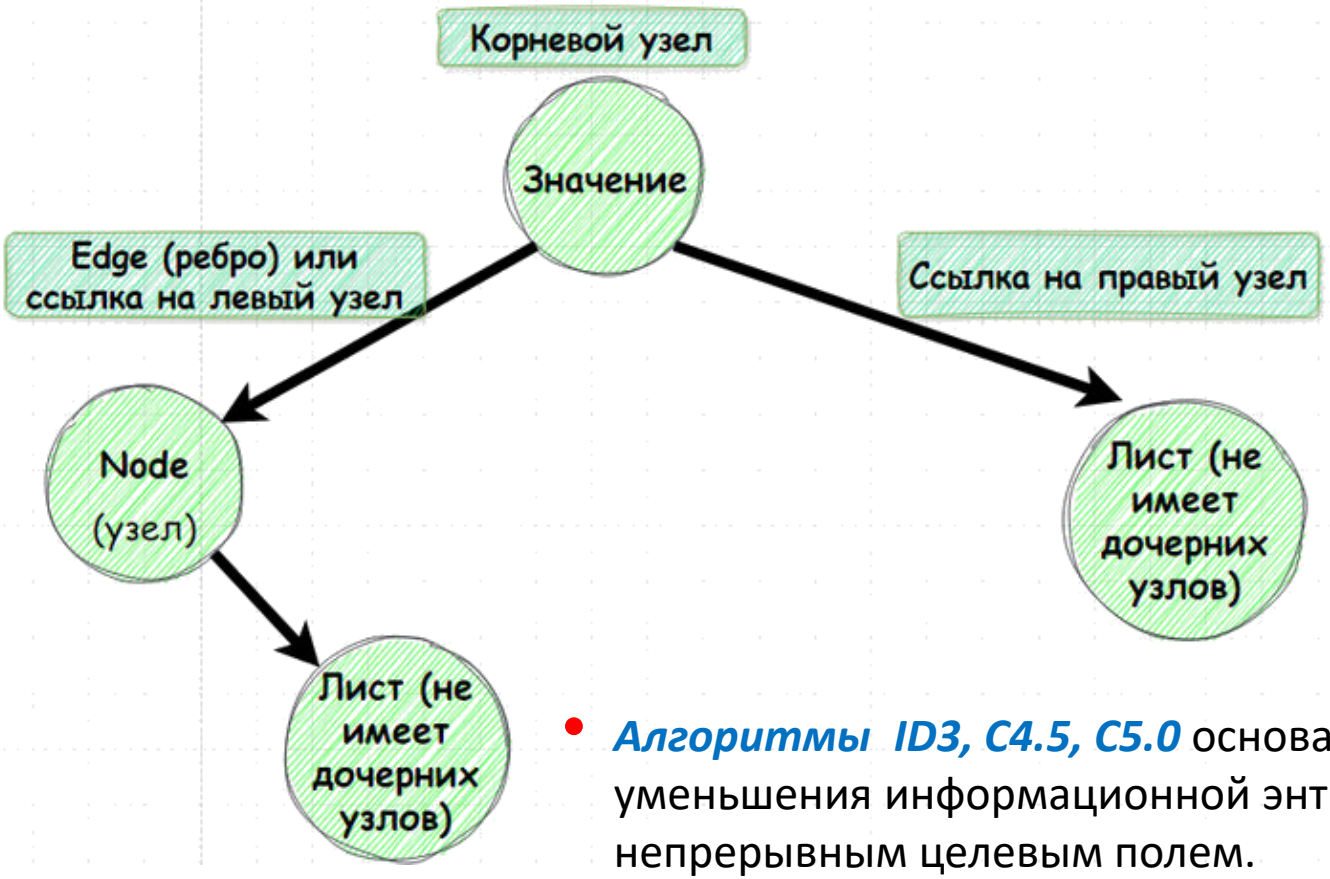
- Дерево решений – это дерево:
  - ***В узлах***, не являющихся листьями (крайними) находятся атрибуты, по которым различаются объекты множества
  - ***В листьях***: значения целевой функции
  - ***На рёбрах***: значение атрибута, из которого исходит ребро

- ***Корневой узел*** – самый верхний узел в дереве решений.

- ***Лист / терминальный узел*** – узел, который не разделяется на другие узлы.

- Чтобы классифицировать новый объект нужно спуститься по дереву решений до листа и выдать соответствующее значение.

Дерево решений: алгоритмы



- ***Алгоритмы ID3, C4.5, C5.0*** основаны на критерии уменьшения информационной энтропии. Не работает с непрерывным целевым полем.
- ***CART (Classification and Regression Tree)*** –дихотомическая классификационная модель. (у узла только 2 потомка). Решает задачи классификации и регрессии.

Дерево решений: правила разбиения

- ✓ **Задача:** на каждом узле необходимо найти такое условие, которое разбивает множество, ассоциированное с этим узлом, на подмножества в соответствии со значением одного из атрибутов.

- **Общее правило для выбора атрибута:** выбранный атрибут должен разбить множество так, чтобы получаемые в итоге подмножества состояли из объектов, принадлежащих к одному классу, или были максимально приближены к этому.

  - ***Теоретико-информационный критерий*** основан на идеи уменьшения информационной энтропии множества при каждом разделении. Алгоритмы ID3, C4.5, C5.0.

  - ***Статистический критерий*** основан на оценке расстояния между «распределениями» классов (Gini). Алгоритм CART.

$$Gini(c) = 1 - \sum_i p_i^2$$

где p_i вероятность присутствия объектов класса i в текущем узле c

Дерево решений: информационный критерий

- Информационная энтропия — это мера случайности или неопределенности. Уровень энтропии колеблется от 0 до 1. Когда энтропия равна 0, это означает, что подмножество чистое, то есть в нем нет случайных элементов. Когда энтропия равна 1, это означает высокую степень случайности.

- Есть множество ***T*** из ***n*** объектов, ***m*** из которых обладают атрибутом ***S***. Информационная энтропия множества ***T*** по отношению к атрибуту ***S*** равна:

$$H(T,S) = -\frac{m}{n}\log_2\left(\frac{m}{n}\right) - \frac{n-m}{n}\log_2\left(\frac{n-m}{n}\right)$$

- Если атрибут ***S*** не бинарный, а может принимать s различных значений, каждое из которых реализуется в m_i случаях, то

$$H(T,S) = -\sum_{i=1}^{s}\frac{m_i}{n}\log_2\left(\frac{m}{n}\right)$$

Дерево решений: информационный критерий

- ✓ Атрибут классификации нужно выбирать так, чтобы после разделения информационная энтропия (относительно целевого функции) стала как можно меньше (или больше прирост информации).

- ✓ Прирост информации при разбиении рассчитывается путем вычитания энтропий каждой ветви из исходной энтропии. Используем его для принятия решения о порядке расположения атрибутов (признаков, свойств) в узлах дерева решений.

Прирост информации = Энтропия корневого узла − Энтропия наследников

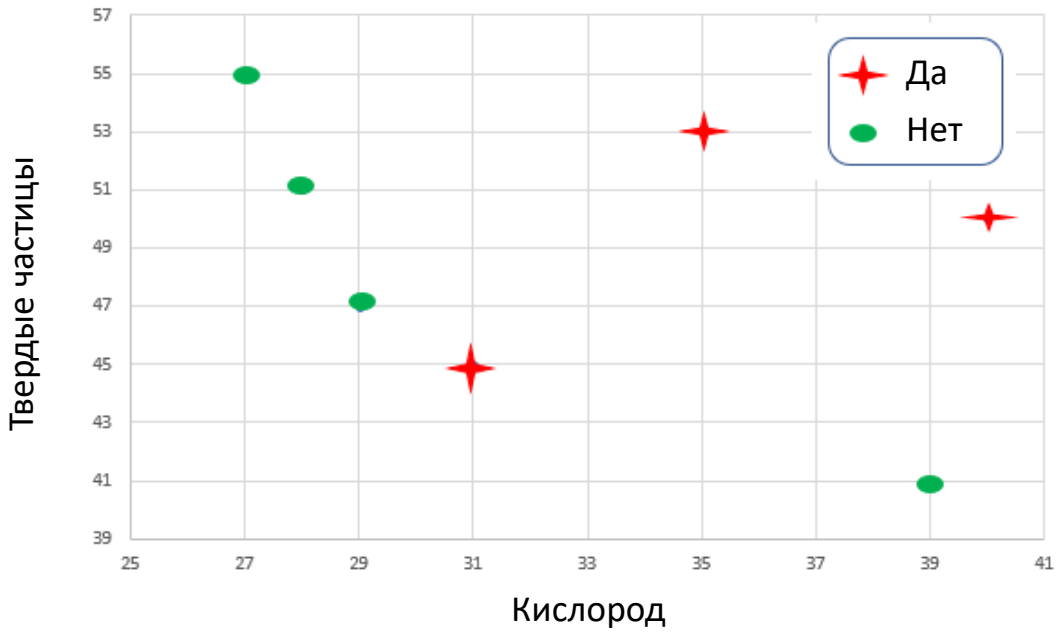
$$Gain(T,S) = H(T) - \sum_{i \in S} \frac{|T_i|}{T} H(T_i)$$

$H(T)$ - энтропия множества T относительно свойства S
S- свойство (атрибут) объекта из множества T
i – возможные значения свойства S
T_i – подмножество, возникающее в результате разделения

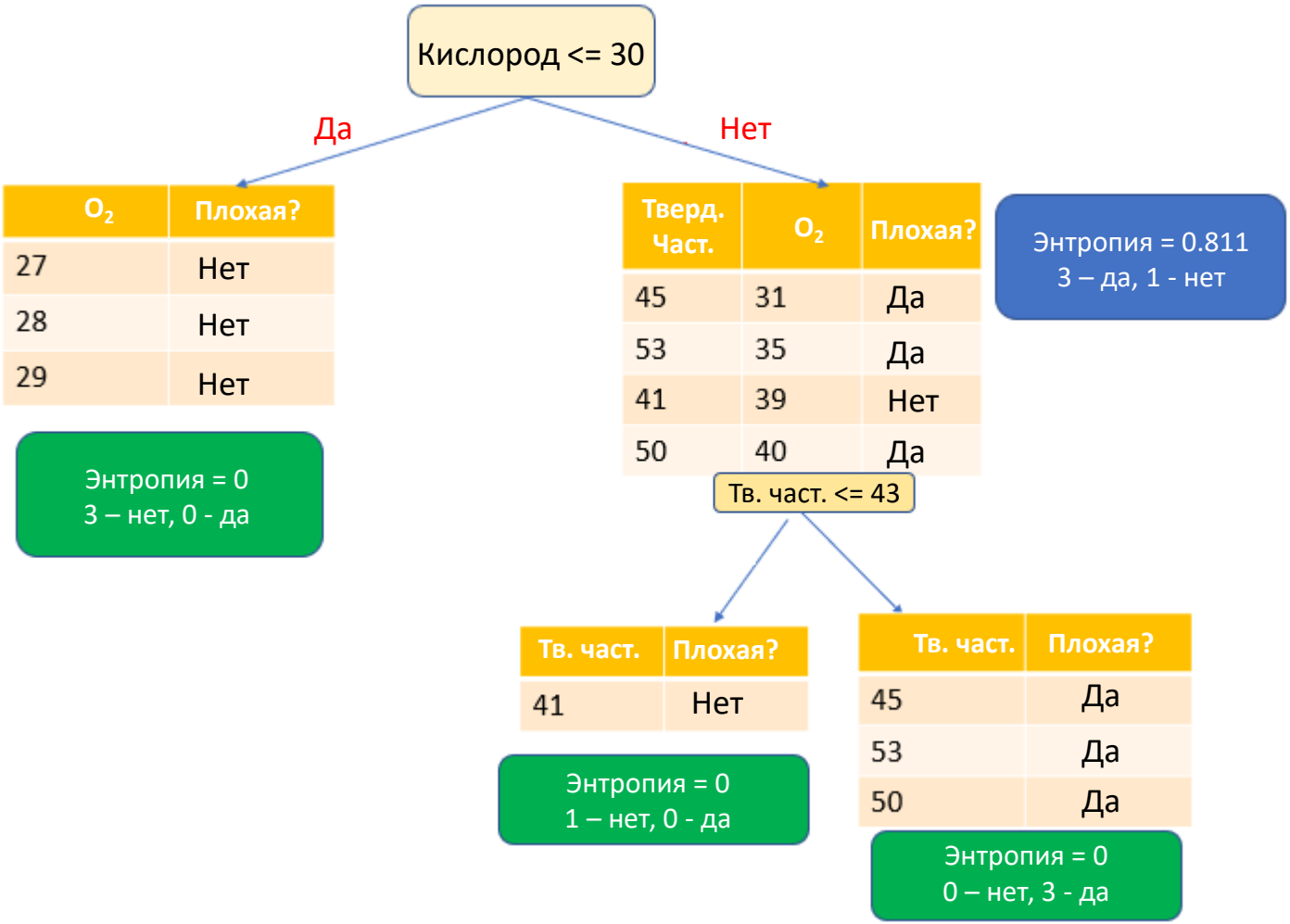
Дерево решений, как водичка?

- Есть данные по насыщению кислородом (в %) и концентрации твердых частиц (в мг/л) для пригодной и непригодной воды.

Кислород	Твердые частицы	Плохая?
31	45	Да
29	47	Нет
27	55	Нет
35	53	Да
28	51	Нет
40	40	Да
39	41	Нет



Дерево решений, как водичка?



Дерево решений, как водичка?

```python
# Импортируем библиотеки
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Загружаем данные
df=pd.read_csv("water.csv")

# Разделяем перменные на x и y
# Атрибуты (кислород и твердые частицы - это x)
# Целевая переменная (плохая?) - это y

x=df.iloc[:,:2] # столбцы до 2 - это x
y=df.iloc[:,2:] # столбцы с 2 идалее - это y

# Строим модель с помощью sklearn
from sklearn import tree
model=tree.DecisionTreeClassifier(criterion="entropy")
model.fit(x,y)

# Оценка модели
model.score(x,y)
```

```python
# Прогнозирование с помощью модели

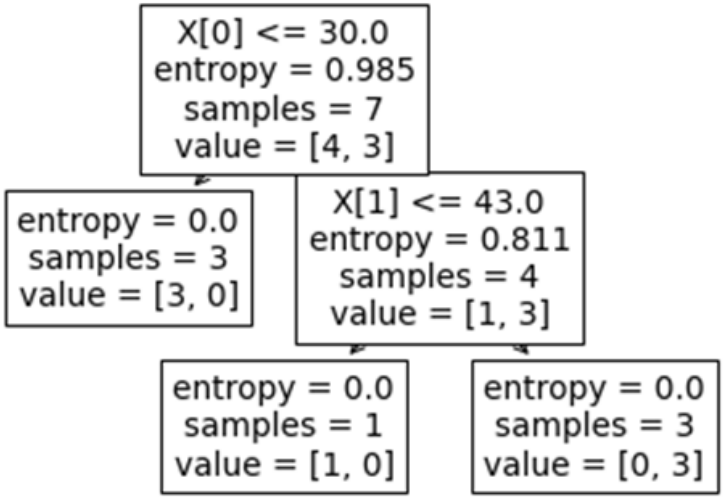
# Предскажем плохая ли вода, если
# содержание твердых частиц 47 мг/л,
# а нысщенность кислородом 29%
model.predict([[29,47]])
# Вывод:
array(['no'], dtype=object)
```

```python
# Предскажем плохая ли вода, если
# содержание твердых частиц 47 мг/л,
# а нысщенность кислородом 45%
model.predict([[45,47]])
# Вывод:
array(['yes'], dtype=object)
```

Дерево решений, как водичка?

```
41  # Визуализация модели
42  tree.plot_tree(model)
```

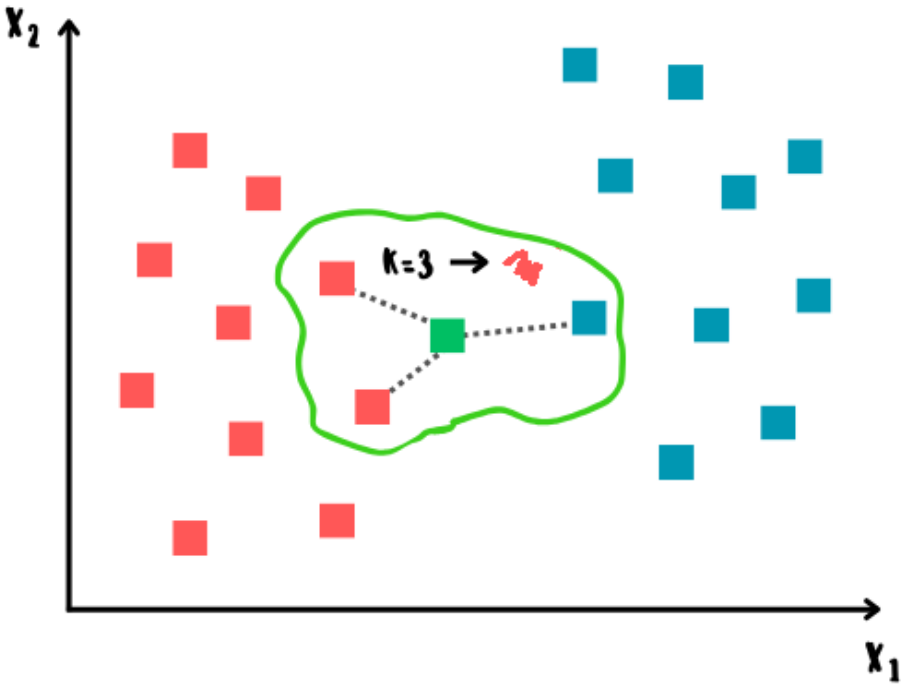
```
[Text(133.92000000000002, 181.2, 'X[0] <= 30.0\nentropy = 0.985\nsamples = 7\nvalue = [4, 3]'),
 Text(66.96000000000001, 108.72, 'entropy = 0.0\nsamples = 3\nvalue = [3, 0]'),
 Text(200.88000000000002, 108.72, 'X[1] <= 43.0\nentropy = 0.811\nsamples = 4\nvalue = [1, 3]'),
 Text(133.92000000000002, 36.23999999999998, 'entropy = 0.0\nsamples = 1\nvalue = [1, 0]'),
 Text(267.84000000000003, 36.23999999999998, 'entropy = 0.0\nsamples = 3\nvalue = [0, 3]')]
```



K-ближайших соседей

- ***k-ближайших соседей (k-Nearest Neighbors или просто kNN)*** – это алгоритм классификации, основанный на гипотезе компактности, которая предполагает, что расположенные близко друг к другу объекты в пространстве признаков имеют схожие значения целевой переменной или принадлежат к одному классу.

1. Вычисляем расстояние между тестовым и всеми обучающими объектами.
2. В пространстве свойств выбираем k-ближайших объектов.
3. Вычисляем расстояние до k-ближайших объектов
4. Итоговым прогнозом будет мода (с учетом веса)
5. Повторяем шаги 1-4 для всех объектов



https://habr.com/ru/articles/801885/

К-ближайших соседей: как вычислять расстояние?

- **Евклидово расстояние** — это наиболее простая метрика, которая определяется как длина отрезка между двумя объектами ***a*** и ***b*** в пространстве с ***n*** признаками

$$d(a,b) = \sqrt{\sum_{i=1}^{n}(a_i - b_i)^2}$$

- **Манхэттенское расстояние** — метрика, которая определяется как сумма модулей разностей координат двух точек в пространстве между двумя объектами ***a*** и ***b*** с ***n*** признаками

$$d(a,b) = \sum_{i=1}^{n}|a_i - b_i|$$

- **Косинусное расстояние** — метрика, которая определяется как угол между двумя векторами ***a*** и ***b*** в пространстве с ***n*** признаками

$$d(a,b) = \frac{\sum_{i=1}^{n} a_i b_i}{\sqrt{\sum_{i=1}^{n} a_i^2}\sqrt{\sum_{i=1}^{n} b_i^2}}$$

К-ближайших соседей: нормализация

- ✓ Разные свойства объектов обычно обладают разным диапазоном представленных значений в выборке. Поэтому значения расстояния могут сильно зависеть от атрибутов с бóльшими диапазонами.

- ✓ Поэтому данные в большинстве случаев необходимо нормализовать, чтобы значения всех атрибутов находились в интервале от 0 до 1.

- *MinMax-нормализация*

$$x' = \frac{x - \min[X]}{\max[X] - \min[X]}$$

- *Z-нормализация*

$$x' = \frac{x - \mathrm{M}[X]}{\sigma[X]}$$

$\mathrm{M}[X]$ – среднее значение X, а $\sigma[X]$ – среднеквадратичное отклонение

K-ближайших соседей: оптимальное k / + и -

✓ Нет способа заранее определить наилучшее значение для ***k***, поэтому нужно попробовать несколько значений, чтобы найти лучшее из них. Чаще всего наиболее предпочтительным значением для ***k*** является 5.

- Низкое значение k может привести к эффекту недообучения модели
- При большом значении k возможны трудности с производительностью модели, а также повышается риск переобучения

\+ Устойчивость к выбросам и аномальным значениям

\+ Простая программная реализация

\+ Результаты работы алгоритма легко поддаются интерпретации. Логика работы алгоритма понятна экспертам в различных областях.

\- Данный метод не создает каких-либо моделей, обобщающих предыдущий опыт

\- При классификации объекта используются все доступные данные, поэтому метод KNN является достаточно затратным в вычислительном плане

К-ближайших соседей: sklearn реализация

```python
# Импорт библиотек
import numpy as np
import matplotlib.pyplot as plt
from mlxtend.plotting import plot_decision_regions
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, r2_score
from sklearn.datasets import load_iris, load_diabetes
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor

# Загрузка данных
X1, y1 = load_iris(return_X_y=True, as_frame=True)

# Разделяем данные на обучающую и тестовую выборки
X1_train, X1_test, y1_train, y1_test = train_test_split(X1.values, y1.values, random_state=0)

# Обучение моделей и оценка полученных результатов
sk_knn_clf = KNeighborsClassifier()
sk_knn_clf.fit(X1_train, y1_train)
sk_knn_clf_pred_res = sk_knn_clf.predict(X1_test)
sk_knn_clf_accuracy = accuracy_score(y1_test, sk_knn_clf_pred_res)

print(f'sk KNN classifier accuracy: {sk_knn_clf_accuracy:}')
print(sk_knn_clf_pred_res)

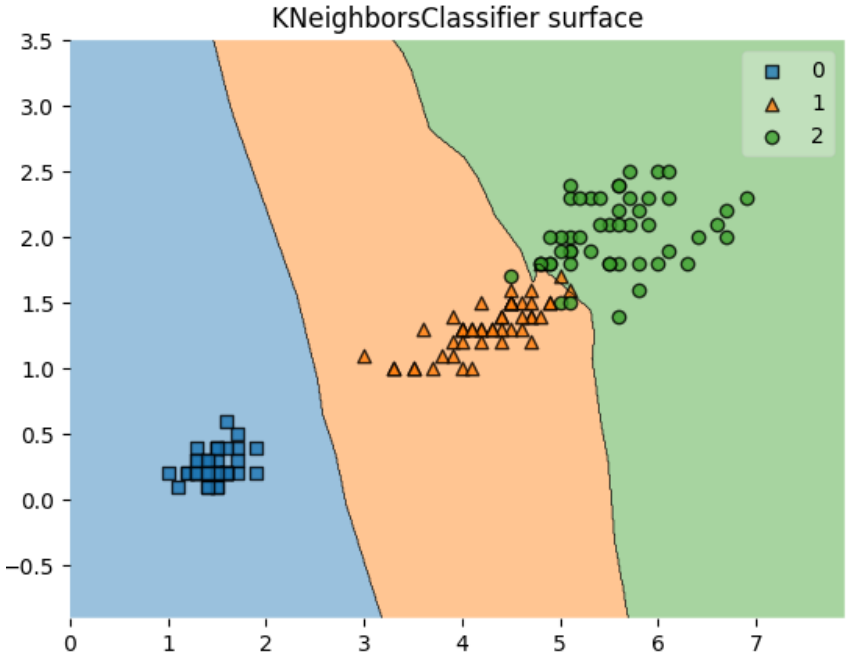
feature_indexes = [2, 3]
title1 = 'KNeighborsClassifier surface'
decision_boundary_plot(X1, y1, X1_train, y1_train, sk_knn_clf, feature_indexes, title1)

sk KNN classifier accuracy: 0.9736842105263158
[2 1 0 2 0 2 0 1 1 1 2 1 1 1 1 0 1 1 0 0 2 1 0 0 2 0 0 1 1 0 2 1 0 2 2 1 0 2]
```

К-ближайших соседей: sklearn реализация

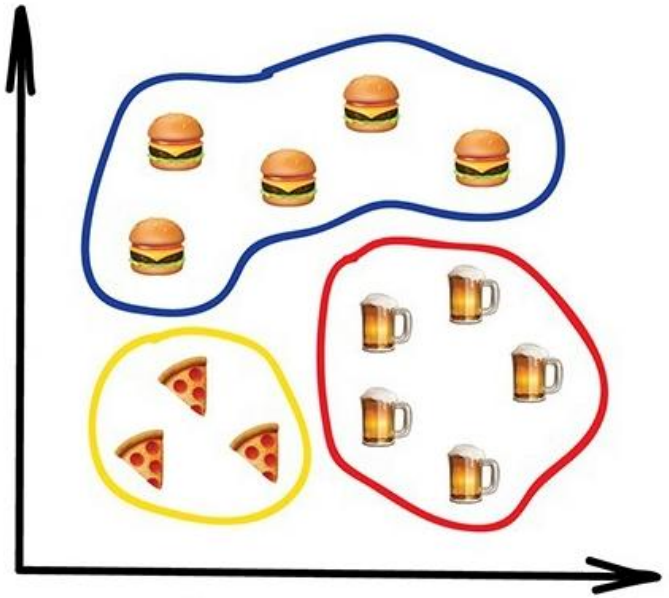
```
def decision_boundary_plot(X, y, X_train, y_train, clf, feature_indexes, title=None):
    feature1_name, feature2_name = X.columns[feature_indexes]
    X_feature_columns = X.values[:, feature_indexes]
    X_train_feature_columns = X_train[:, feature_indexes]
    clf.fit(X_train_feature_columns, y_train)

    plot_decision_regions(X=X_feature_columns, y=y.values, clf=clf)
    plt.xlabel(feature1_name)
    plt.ylabel(feature2_name)
    plt.title(title)
```



Кластеризация

- Основная идея кластерного анализа (clustering, cluster analysis) заключается в том, чтобы разбить объекты на группы или кластеры таким образом, чтобы внутри группы эти наблюдения были более похожи друг на друга, чем на объекты другого кластера.



- Заранее не известно на какие кластеры необходимо разбить данные. Обучение модели происходит на неразмеченных данных, то есть без целевой переменной.

- Популярные методы:
  - Метод K-средних
  - Mean-Shift
  - DBSCAN

- Часто применяется для анализа и разметки новых данных (подготовка обучающих данных для модели классификатора)

Метод K-средних

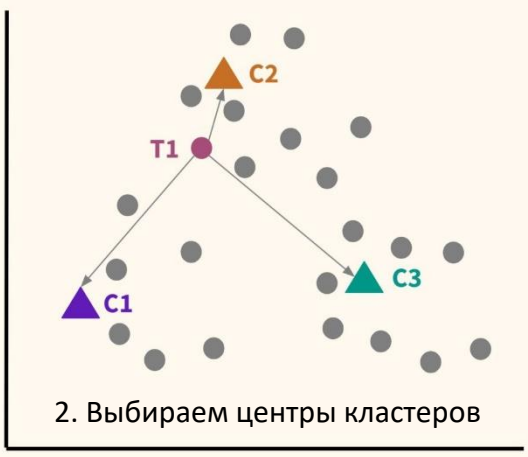
- ***Метод k-средних*** используется для кластеризации данных на основе алгоритма разбиения векторного пространства на заранее определенное число кластеров ***k***

1. Выбирается число кластеров ***k***.
2. Из исходного множества данных случайно выбираются ***k*** объектов (или произвольных точек), которые служат начальными центрами кластеров.
3. Для каждого объекта исходного множества определяется ближайший к нему центр кластера (в метрике Евклида). Объекты, «притянутые» определенным центром, образуют кластеры.
4. Вычисляются центроиды — центры тяжести кластеров. Каждый центроид — это вектор, элементы которого представляют собой средние значения соответствующих признаков, вычисленные по всем объектам кластера.
5. Центр кластера смещается в его центроид, после чего центроид становится центром нового кластера.
6. Оцениваем качество каждого кластера как сумму квадратов внутрикластерных расстояний до центра кластера (WCSS).
7. Шаги 3-6 итеративно повторяются до сходимости по структуре кластеров.

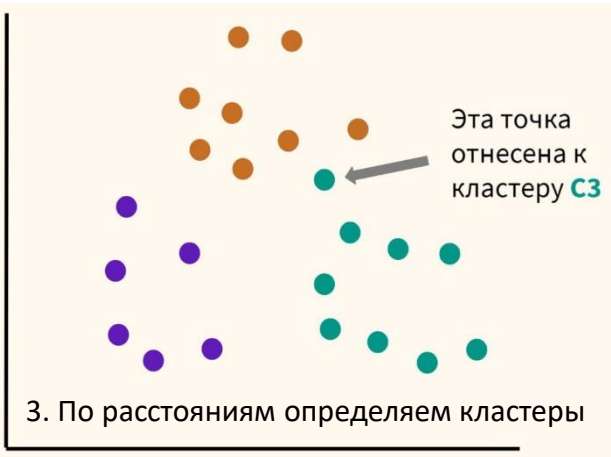
Метод K-средних



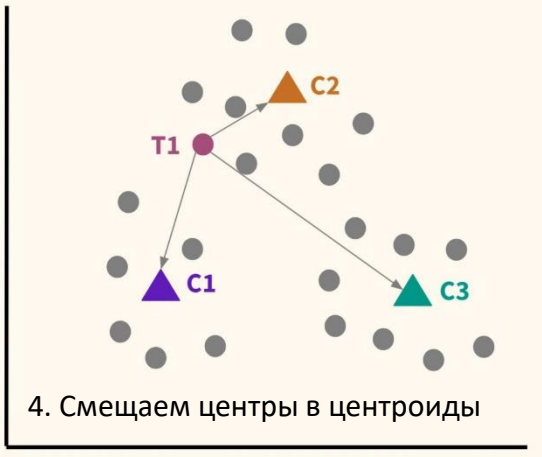
1. Выбираем количество кластеров



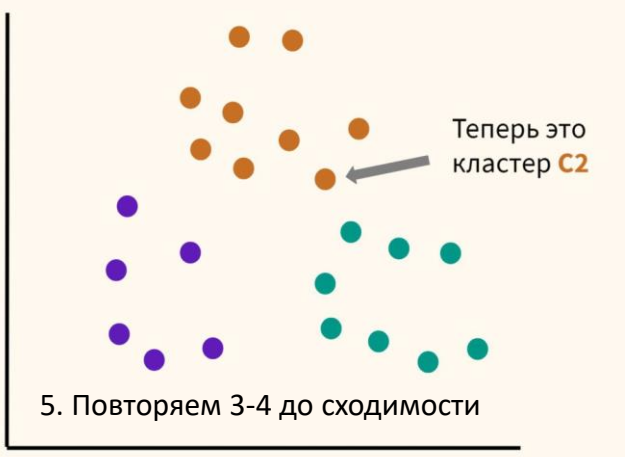
2. Выбираем центры кластеров



3. По расстояниям определяем кластеры

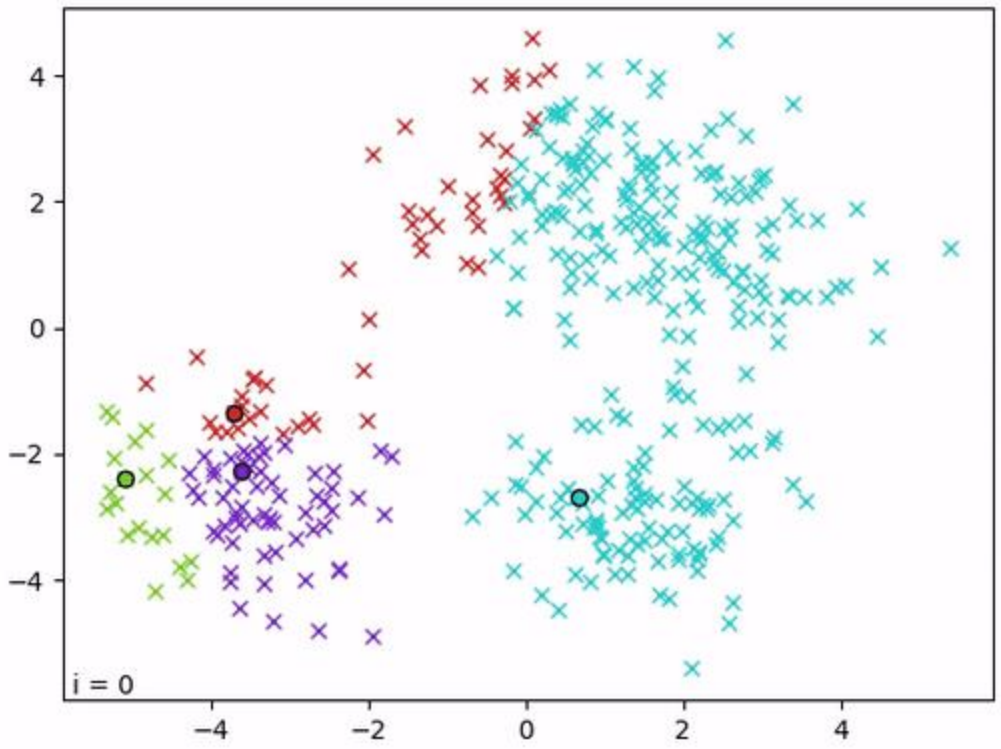


4. Смещаем центры в центроиды



5. Повторяем 3-4 до сходимости

Метод К-средних

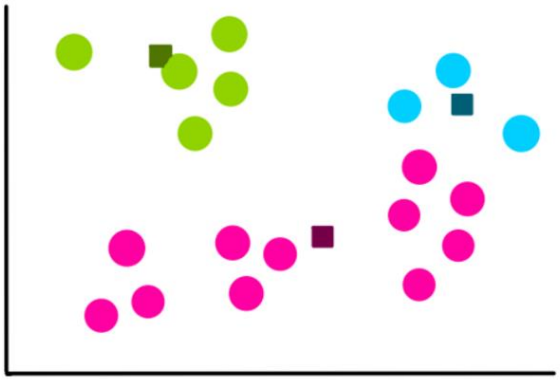


Метод K-средних: оценка качества кластеров

- «Качество» кластера измеряется суммой квадратов расстояний от объекта, принадлежащего данному кластеру, до центроида этого кластера.

- **Основная идея:** определить такие кластеры, что расстояния внутри каждого кластера минимальны. Чтобы оценить расстояния, вычисляется величина *WCSS* (within-cluster sum of squares) – *сумма квадратов внутрикластерных расстояний до центра кластера*

$$J = WCSS = \sum_{j=1}^{k} \sum_{i=1}^{n} \left\| x_i^j - c_j \right\|^2$$



k – количество кластеров

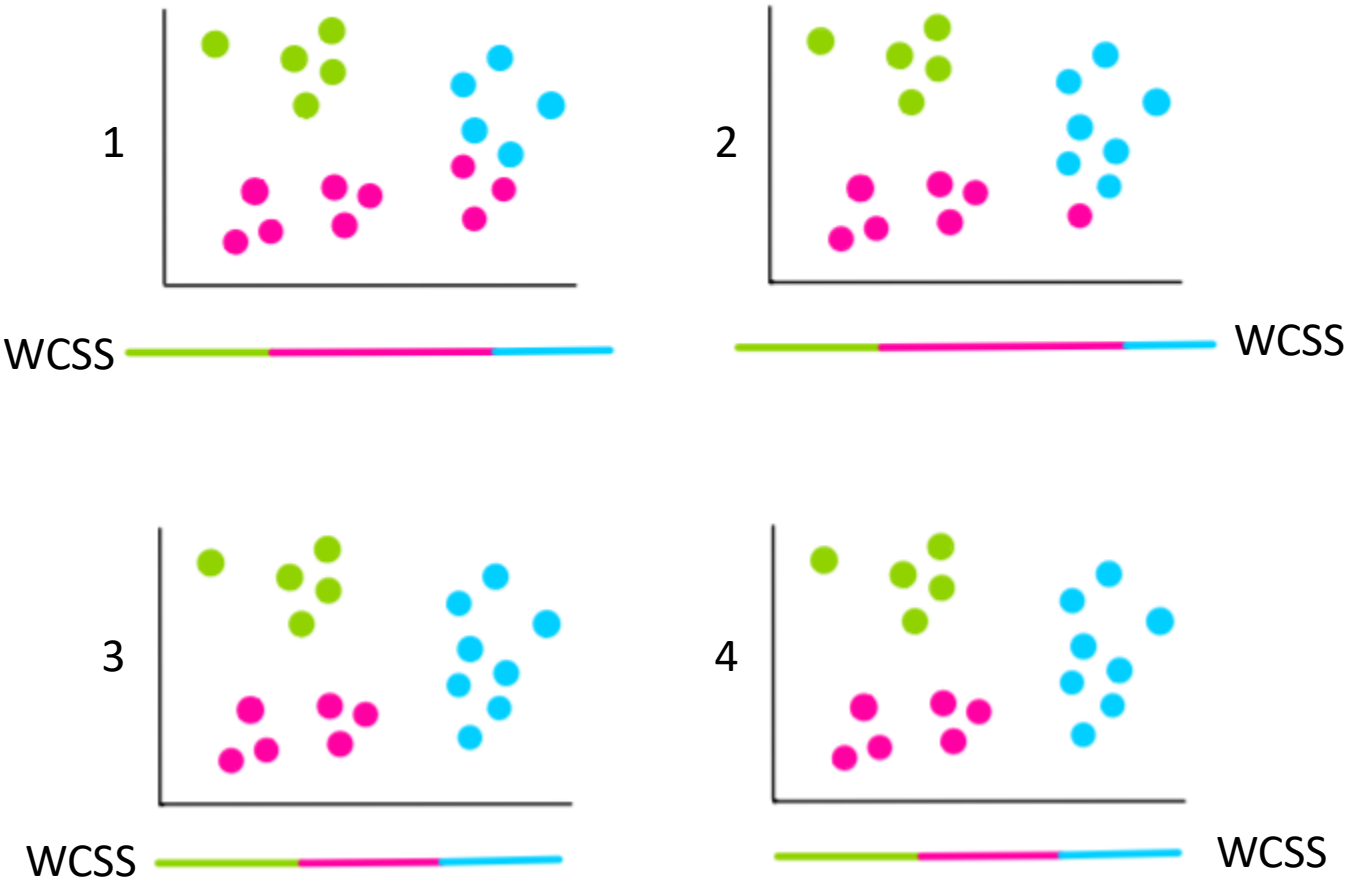
n – количество объектов в данном кластере

x_i^j - значение признака x в i-ом объекте j-ого кластера

c_j - значение признака x в центроиде j-ого кластера

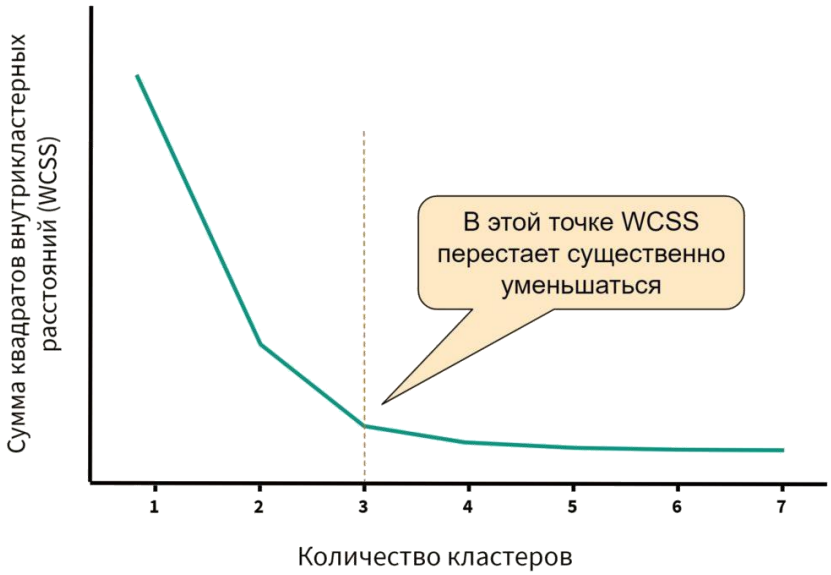
Метод K-средних: сходимость по WCSS

$$J = WCSS = \sum_{j=1}^{k} \sum_{i=1}^{n} \left\| x_i^j - c_j \right\|^2$$

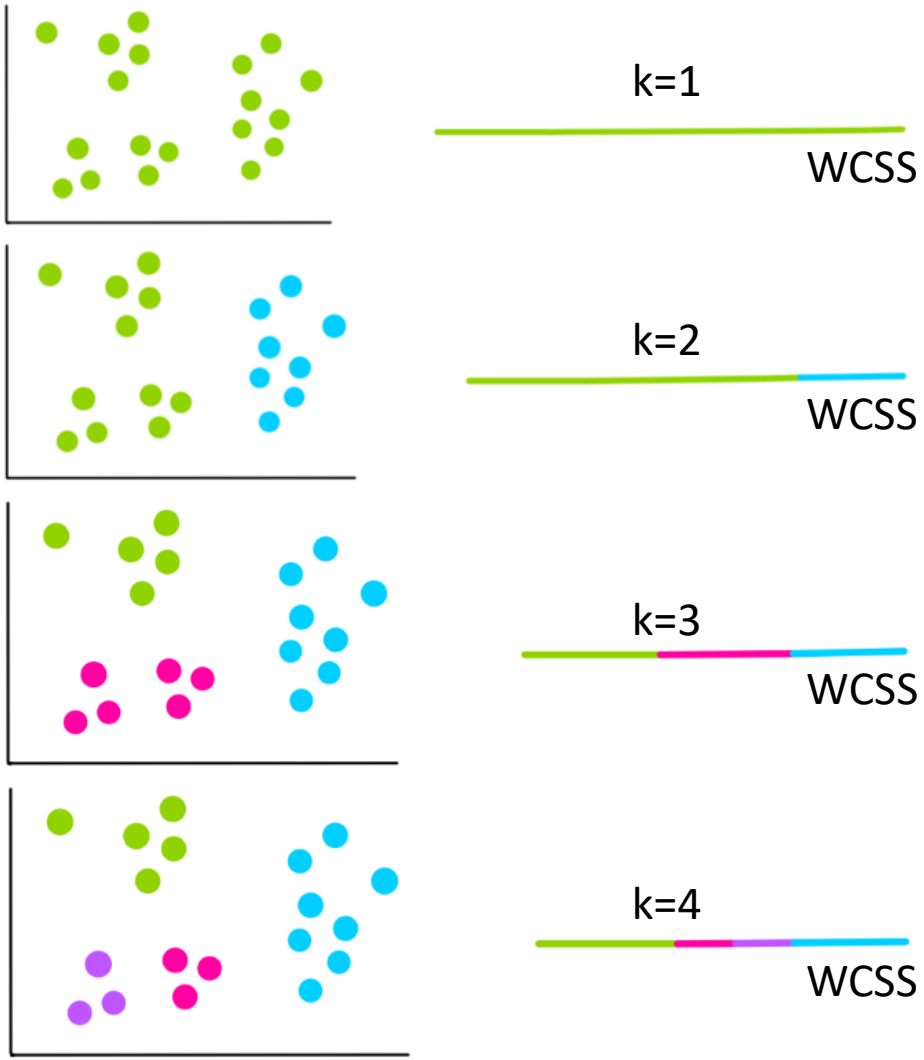


Метод K-средних: выбор количества кластеров

- ✓ Оптимальное количество кластеров определяется сходимостью по величине WCSS



$$J = WCSS = \sum_{j=1}^{k} \sum_{i=1}^{n} \left\| x_i^j - c_j \right\|^2$$



Метод K-средних: реализация sklearn

```
# вначале импортируем библиотеки
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

"""### Этап 1. Загрузка данных"""

# импортируем набор данных из модуля datasets библиотеки sklearn
from sklearn.datasets import load_iris
iris = load_iris()

# создаем датафрейм, берем данные из iris.data и название столбцов из iris.feature_names
iris_df = pd.DataFrame(iris.data, columns = iris.feature_names)

# смотрим первые пять значений
iris_df.head()

# посмотрим на размерность
iris_df.shape
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```
(150, 4)
```

Метод K-средних: реализация sklearn

```
"""### Этап 2. Предаварительная обработка данных"""

# проверим, есть ли пропущенные значения
iris_df.isnull().sum()

# импортируем необходимый класс из модуля preprocessing библиотеки sklearn
from sklearn.preprocessing import StandardScaler

# создадим объект этого класса
scaler = StandardScaler()

# приведем данные к единому масштабу
iris_scaled = scaler.fit_transform(iris_df)

# заново создадим датафрейм с нормализованными значениями
iris_df_scaled = pd.DataFrame(iris_scaled, columns = iris.feature_names)

# посмотрим на первые пять строк
iris_df_scaled.round(2).head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	-0.90	1.02	-1.34	-1.32
1	-1.14	-0.13	-1.34	-1.32
2	-1.39	0.33	-1.40	-1.32
3	-1.51	0.10	-1.28	-1.32
4	-1.02	1.25	-1.34	-1.32

```
"""### Этап 3. EDA и отбор признаков"""

# оставим все признаки и для наглядности
# поместим наши данные в переменную X
# целевой переменной у нас разумеется нет
X = iris_df_scaled
X.head()
```

Метод K-средних: реализация sklearn

```
"""### Этап 4. Обучение и оценка качества модели

Выбор количества кластеров (настройка гиперпараметра)
"""

# импортируем нужный класс
from sklearn.cluster import KMeans

# создадим пустой список для записи показателя WCSS (нашей ошибки)
wcss = []

# воспользуемся новой для нас функцией range(), она возвращает последовательность чисел
# в данном случае числа от 1 до 10
for i in range(1, 11):

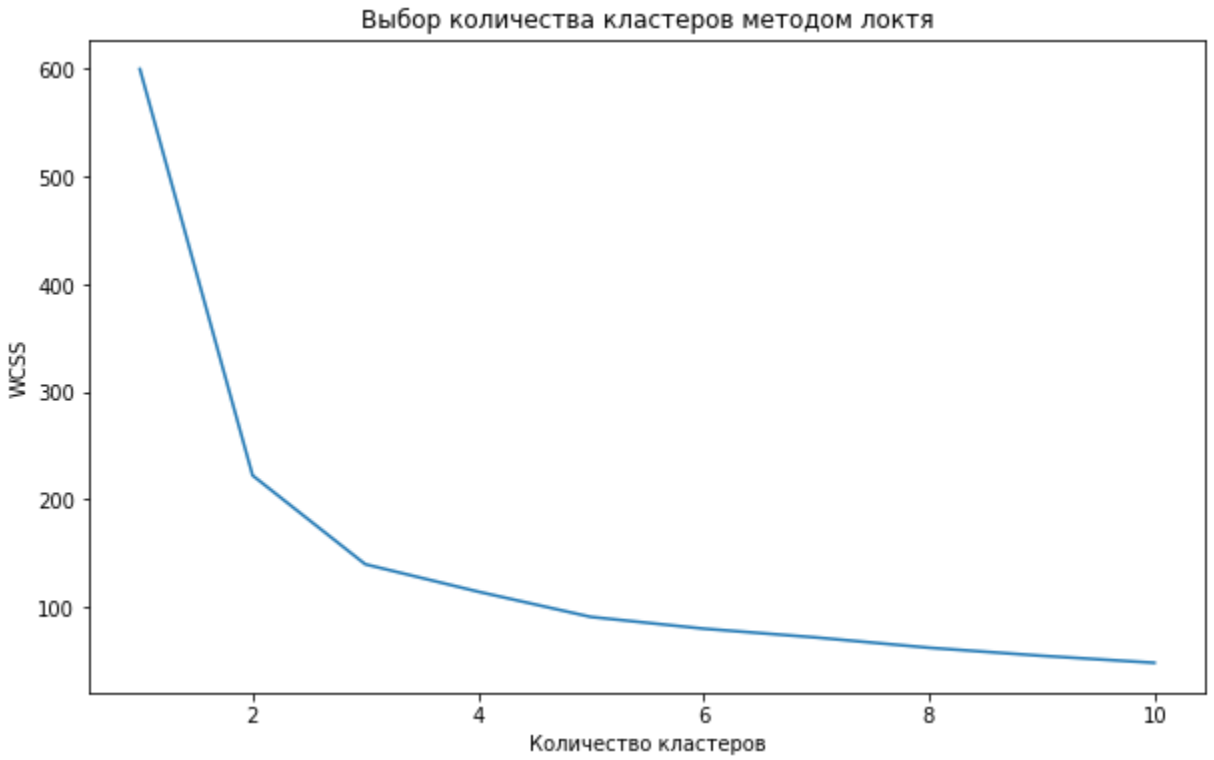
    # настроим параметры модели (подробнее об этом на сайте)
    kmeans = KMeans(n_clusters = i, init = 'k-means++', max_iter = 300, n_init = 10, random_state = 42)

    # обучим модель на наших данных с разным количеством кластеров
    kmeans.fit(X)

    # для каждого кластера рассчитаем ошибку (атрибут inertia_) и поместим в список
    wcss.append(kmeans.inertia_)

# зададим размер графика
plt.figure(figsize = (10,6))
# передадим функции plot() последовательность кластеров и ошибку WCSS
plt.plot(range(1, 11), wcss)
# и добавим подписи
plt.title('Выбор количества кластеров методом локтя')
plt.xlabel('Количество кластеров')
plt.ylabel('WCSS')
```

Метод K-средних: реализация sklearn



Метод K-средних: реализация sklearn

```python
"""Оценка качества"""

# посмотрим на WCSS
kmeans.inertia_

# посмотрим на нашу целевую переменную (такое возможно только в учебном датасете!)
print(iris.target)

# теперь на прогноз
print(y_pred)

# мы видим, что кодировка видов не совпадает

# пройдемся списком по всему массиву y_pred
for i in range(len(y_pred)):
  # если было 1, заменим на 0
  if y_pred[i] == 1:
    y_pred[i] = 0
  # если было 0, будет 1
  elif y_pred[i] == 0:
    y_pred[i] = 2
  # если было 2, будет 1
  elif y_pred[i] == 2:
    y_pred[i] = 1

# посмотрим на измененную кодировку
print(y_pred)
```

	Target	Prediction	Comparison
0	0	0	True
1	0	0	True
2	0	0	True
3	0	0	True
4	0	0	True

```python
# поместим переменные в словарь
d = {'Target' : iris.target, 'Prediction' : y_pred}

# превратим словарь в датафрейм с соответствующими названиями столбцов
result = pd.DataFrame(d, columns = ['Target', 'Prediction'])
result.head(2)

# с помощью функции where зададим логическое значение
# если совпадает, True, если нет - False
comparison = np.where(result['Target'] == result['Prediction'], True, False)

# на выходе получим массив Numpy
print(type(comparison))
print(comparison[:5])

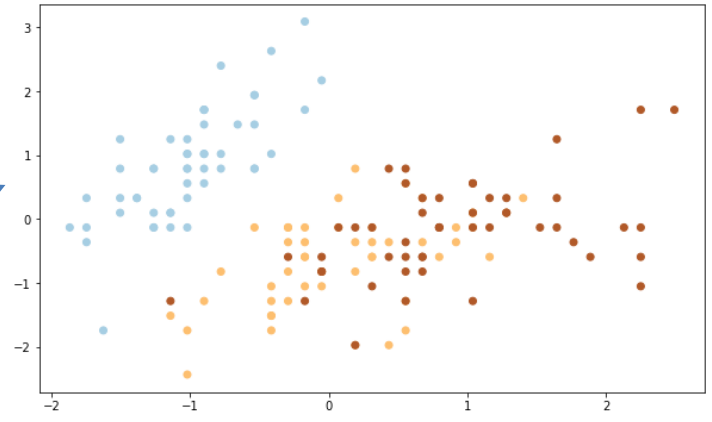
# добавим этот массив в качестве столбца в датафрейм
result['Comparison'] = comparison
result.head()

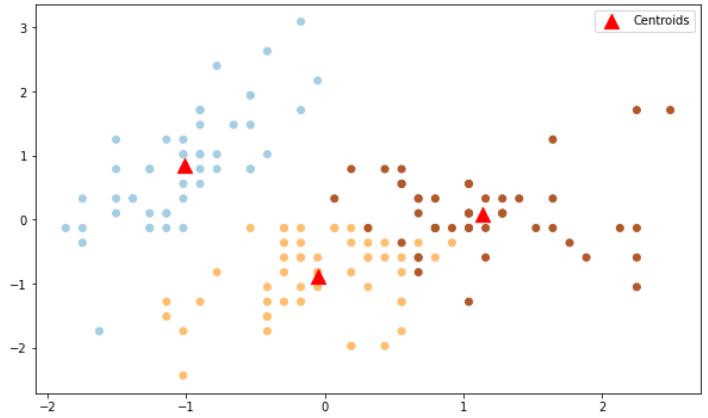
# выведем долю совпавших (True) и не совпавших (False) значений
result['Comparison'].value_counts(normalize = True).round(2)
```

```
True    0.83
False   0.17
Name: Comparison, dtype: float64
```

Метод K-средних: реализация sklearn

```
151  # посмотрим на целевую переменную с помощью точечной диаграммы
152
153  # зададим размер
154  plt.figure(figsize = (10,6))
155
156  # возьмем первый и второй столбцы с помощью функции iloc()
157  # параметр c задаст каждой точке определенный код (0, 1, 2), а
158  # camp определит цвет каждой из точек
159  plt.scatter(X.iloc[:,0], X.iloc[:,1], c = iris.target, cmap = 'Paired')
160
161  # теперь посмотим на результат алгоритма кластеризации
162  plt.figure(figsize = (10,6))
163
164  # снова построим точечную диаграмму, но коды точек возьмем из y_pred
165  plt.scatter(X.iloc[:,0], X.iloc[:,1], c = y_pred, cmap='Paired')
166
167  # кроме того выведем центроиды с помощью атрибута cluster_centers_
168  # зададим размер s = 150, красный цвет c = 'red', тип маркера - треугольник, и подпись
169  plt.scatter(kmeans.cluster_centers_[:,0], kmeans.cluster_centers_[:,1], s = 150, c = 'red', marker = '^', label = 'Centroids')
170
171  # укажем, где разместить легенду
172  plt.legend(loc = 'upper right')
```





✓ **Резюме:** При решении задач кластеризации мы берем данные, обязательно их масштабируем и выбираем количество кластеров (с помощью экспертной оценки или метода локтя). К сожалению, дать точную оценку качества кластеризации бывает очень сложно из-за отсутствия разметки.

Классическое машинное обучение для ХТ: Резюме

Вопрос	Алгоритм	Примеры
Это А или Б?	Классификация	Классификация объектов, процессов на два и более класса
Это странно?	Классификация Кластеризация Регрессия	Оценка рисков (отказ аппарата, выхода продукта плохого качества и т.п.)
Сколько?	Регрессия	Выход целевого продукта, расход теплоносителя, степень обессеривания бензина
Как это организовано?	Кластеризация	*Рекомендации* по условиям проведения реакции, по тех. режиму установки или аппарата
Что делать дальше?	Обучение с подкреплением	Системы управления (поддержание температуры, давления, расхода и т.п.)